```
import numpy as np
import networkx as nx

class AntColonyOptimizationNetwork:
    def __init__(self, graph, n_ants=10, n_iterations=50, alpha=1.0, beta=2.0, rho=0.1, Q=1.0):
        self.graph = graph
        self.nodes = list(graph.nodes())
        self.n_ants = n_ants
        self.n_iterations = n_iterations
        self.alpha = alpha  # Pheromone weight
        self.beta = beta    # Heuristic (distance/cost) weight
        self.rho = rho      # Pheromone evaporation rate
        self.Q = Q          # Pheromone deposit factor

        # Initialize pheromones on all edges
        self.pheromones = {}
        for u, v in self.graph.edges():
            self.pheromones[(u, v)] = 1.0
            self.pheromones[(v, u)] = 1.0

    def run(self, source, destination):
        best_path = None
        best_cost = float('inf')

        for iteration in range(self.n_iterations):
            all_paths = []
            all_costs = []

            for _ in range(self.n_ants):
                path = self._construct_path(source, destination)
                if path:
                    cost = self._get_path_cost(path)
                    all_paths.append(path)
                    all_costs.append(cost)

                    if cost < best_cost:
                        best_cost = cost
                        best_path = path

            # Evaporate pheromones
            for edge in self.pheromones:
                self.pheromones[edge] *= (1.0 - self.rho)

            # Deposit new pheromones
            for path, cost in zip(all_paths, all_costs):
                for i in range(len(path) - 1):
                    u, v = path[i], path[i+1]
                    self.pheromones[(u, v)] += self.Q / cost
                    self.pheromones[(v, u)] += self.Q / cost

        return best_path, best_cost

    def _construct_path(self, source, destination):
        path = [source]
        current_node = source
        visited = set(path)

        while current_node != destination:
            neighbors = list(self.graph.neighbors(current_node))
            valid_neighbors = [n for n in neighbors if n not in visited]

            if not valid_neighbors:
                return None  # Dead end

            # Calculate transition probabilities
            probabilities = []
            for n in valid_neighbors:
                cost = self.graph[current_node][n].get('cost', 1.0)
                tau = self.pheromones.get((current_node, n), 1.0)
                eta = 1.0 / (cost + 1e-10)  # Heuristic: inverse of cost
                prob = (tau ** self.alpha) * (eta ** self.beta)
                probabilities.append(prob)

            prob_sum = sum(probabilities)
            if prob_sum == 0:
                return None

            probabilities = [p / prob_sum for p in probabilities]
            next_node = np.random.choice(valid_neighbors, p=probabilities)
            path.append(next_node)
            visited.add(next_node)
```

```
            visited.add(next_node)
            current_node = next_node

        return path

    def _get_path_cost(self, path):
        total_cost = 0
        for i in range(len(path) - 1):
            total_cost += self.graph[path[i]][path[i+1]].get('cost', 1.0)
        return total_cost


def create_network_from_user():
    G = nx.Graph()
    print("=== Computer Network Setup ===")
    num_edges = int(input("Enter the number of network links (edges): "))
    print("Enter each link in the format: <Node1> <Node2> <Cost/Latency>")
    for _ in range(num_edges):
        parts = input("Link (e.g., A B 2): ").split()
        u, v, cost = parts[0], parts[1], float(parts[2])
        G.add_edge(u, v, cost=cost)

    return G


if __name__ == "__main__":
    # Example input interaction
    network_graph = create_network_from_user()

    source_node = input("Enter Source Node: ").strip()
    dest_node = input("Enter Destination Node: ").strip()

    aco = AntColonyOptimizationNetwork(
        graph=network_graph,
        n_ants=20,
        n_iterations=50,
        alpha=1.0,
        beta=2.0,
        rho=0.1
    )

    path, cost = aco.run(source_node, dest_node)

    print("\n--- Optimization Result ---")
    if path:
        print(f"Shortest Path found: {' -> '.join(path)}")
        print(f"Total Path Cost/Latency: {cost}")
    else:
        print("No valid path found between the source and destination.")
```

```
=== Computer Network Setup ===
Enter the number of network links (edges): 12
Enter each link in the format: <Node1> <Node2> <Cost/Latency>
Link (e.g., A B 2): A B 2
Link (e.g., A B 2): A D 7
Link (e.g., A B 2): A F 8
Link (e.g., A B 2): B C 4
Link (e.g., A B 2): B F 5
Link (e.g., A B 2): C G 13
Link (e.g., A B 2): D F 9
Link (e.g., A B 2): D E 10
Link (e.g., A B 2): E F 7
Link (e.g., A B 2): F G 14
Link (e.g., A B 2): F C 12
Link (e.g., A B 2): E G 17
Enter Source Node: A
Enter Destination Node: G

--- Optimization Result ---
Shortest Path found: A -> B -> C -> G
Total Path Cost/Latency: 19.0
```